



A new DIRECT–GLh algorithm for global optimization with hidden constraints

Linas Stripinis¹ · Remigijus Paulavičius¹

Received: 25 July 2020 / Accepted: 9 March 2021 / Published online: 29 March 2021

© The Author(s), under exclusive licence to Springer-Verlag GmbH Germany, part of Springer Nature 2021

Abstract

In this paper, we consider the solution of global optimization problems involving hidden constraints. We present a novel deterministic derivative-free global optimization algorithm based on our recently introduced DIRECT–GL (Stripinis et al. in Optim Lett. 12(7):1699–1712, 2018). The new algorithm (DIRECT–GLh) incorporates two additional techniques for faster feasibility detection and solution improvement, especially when the solution lies on the border of the a priori unknown feasible region. The algorithm’s potential is demonstrated on 67 problems from the current release of DIRECTLib and 100 Emmmental-type test problems. The numerical comparison reveals the proposed algorithm’s superiority to the considered DIRECT-type approaches for such problems.

Keywords DIRECT-type algorithm · Derivative-free optimization · Hidden constraints

1 Introduction

We consider the solution to the constrained global optimization problem

$$\min_{\mathbf{x} \in D^{\text{feas}}} f(\mathbf{x}), \quad (1)$$

where $f : \mathbb{R}^n \rightarrow \mathbb{R} \cup \{\infty\}$ denotes an extended real-valued, most likely “black-box” objective function. A priori unknown feasible region D^{feas} is defined as a non-empty

✉ Remigijus Paulavičius
remigijus.paulavicius@mif.vu.lt
Linas Stripinis
linas.stripinis@mif.vu.lt

¹ Vilnius University Institute of Data Science and Digital Technologies, Akademijos 4, LT-08663 Vilnius, Lithuania

set

$$D^{\text{feas}} = D \cap D^{\text{hidden}} \neq \emptyset,$$

where D is an n -dimensional hyper-rectangle

$$D = [\mathbf{a}, \mathbf{b}] = \{\mathbf{x} \in \mathbb{R}^n : a_j \leq x_j \leq b_j, j = 1, \dots, n\},$$

and D^{hidden} are not given by explicit formulae hidden constraints. Such a problem formulation leads to a complex and analytically undefined feasible region. Moreover, we assume that the objective and constraints are Lipschitz-continuous functions with unknown Lipschitz constants and can be non-differentiable, nonlinear, nonconvex, and multimodal.

Problems with hidden constraints often occur when the objective function is not everywhere defined [6]. Such situations arise, e.g., when the simulation crashes [26], when computations within the objective function fail [4,7,9,56], or when the evaluation of the objective function itself tests for feasibility conditions that are not given by closed-form inequalities [5,25]. In the same vein as in [6,26], we call these internal to f constraints as *hidden constraints* and assume that f fails to return a value when evaluated at $\mathbf{x} \notin D^{\text{feas}}$. Alternatively, some authors may use other terms like *crash*, *unknown*, *unspecified* and *forgotten* constraints [1,26]. In addition, we note the similarity of the problem (1) to optimization problems with the partially defined objective function and constraints (see, e.g., [3,47,50,52], and the references given therein). If a constraint is not satisfied at a point, the rest of the constraints and the objective function may not be defined at that point [13,44,50,52], and, therefore, it may not be computed.

Derivative-free optimization [43] is the key technique for problems with unavailable, unreliable, or impractical to obtain derivative information. The original DIRECT (DIviding RECTangles) algorithm, developed by Jones et al. [24], is a deterministic derivative-free sampling method for box-constrained global optimization problems. The algorithm is an efficient extension to a classical Lipschitz optimization [36,37,39,41,42,46,48,49,55], without any a priori knowledge of the Lipschitz constant. This property makes a large class of various DIRECT-type methods desirable for the solution of various real-world problems (see, e.g., [2,12,19,29,35,58,60] and the references given therein). Since the original DIRECT algorithm, various DIRECT-type modifications and extensions have been proposed. For a thorough review, we refer the reader to a recent survey [23]. Below, we briefly review and comment on the potential of existing DIRECT-type methods to the problem (1).

We start from DIRECT-type methods for generally constrained problems. In the first method [22], Jones extended the original DIRECT to handle nonlinear inequality constraints using auxiliary functions. Another DIRECT extension, DIRECT-L1 [14], is based on the exact L1 penalty functions [16]. The comparison in [14] showed promising results. Still, the requirement to set penalty parameters for each constraint function in advance is the biggest drawback and can significantly impact the performance [14,27,39,40]. Two more recent penalty function-based approaches (EPGO [11] and DF-EPGO [10]) proposed an automatic update rule for the penalty param-

eters. Another approach, filter-based DIRECT [8], simultaneously minimizes the constraint violations and the objective function. A metamodel-based [17,53,54] algorithm eDIRECT-C [27] partitions the search space in Voronoi cells and separately handles feasible and infeasible cells. Gorodetsky introduced the auxiliary functions and redundancy-free diagonal partitioning strategy based ExDIR-diag algorithm recently in [20]. Finally, in [58], we have extended DIRECT-GL [57] for problems with general constraints by incorporating a two-step selection procedure and a penalty function approach. All these methods, however, require the constraints to be defined over the whole hyper-rectangle D . Therefore, they cannot be directly applied to the problem (1).

As original DIRECT, the largest class of DIRECT-type methods are devoted to box-constrained problems (see, e.g., [28,30,31,33,34,38–40,47,57], and the references given therein). Unfortunately, most of them cannot be directly (without any modifications) applied for the solution of problem (1). Hidden constraints are typically handled by returning NaN or ∞ when the objective function is called at $\mathbf{x} \notin D^{\text{feas}}$. Therefore, using NaN hyper-rectangles with infeasible midpoints would not be subdivided at all. In the case of ∞ (or any other high value), they would be left unexplored as long as there are the same size hyper-rectangles with feasible centers (see Sect. 2.1). To sum up, very few approaches within the DIRECT framework are well-suited for optimization problems with hidden constraints. In Sect. 2, we review them in more detail and highlight the main drawbacks.

Motivated by this, in this work, we introduce a new auxiliary function-based DIRECT-type algorithm (DIRECT-GLh). The new algorithm is an extension of our recently introduced DIRECT-GL [57], specifically designed for problems with hidden constraints. i.e., does not require any explicit information about the constraint functions.

The rest of the paper is organized as follows. A review of DIRECT-type extensions for global optimization problems with hidden constraints is given in Sect. 2. In Sect. 3, we describe our new DIRECT-GLh approach. An extensive experimental analysis on 167 test problems is presented in Sect. 4. Finally, in Sect. 5, we conclude the paper.

2 DIRECT-type extensions for global optimization with hidden constraints

In this section, we review DIRECT-type extensions for optimization problems with hidden constraints. We note that all these approaches could also be used for problems with general constraints. However, as they do not attempt to capture tangent directions or use constraint functions, they are slower than approaches specifically created for general problems.

2.1 DIRECT extension based on a barrier approach

The first and most natural DIRECT-type extension to handle the problem (1) is based on the barrier approach, which assigns a very high value to infeasible points [18] (let

us call this approach **DIRECT-Barrier**). **DIRECT-Barrier** guarantees that no infeasible hyper-rectangles will be subdivided as long as there are the same size hyper-rectangles with the feasible midpoint. Unfortunately, this straightforward strategy has at least two apparent drawbacks. Hyper-rectangles with an infeasible midpoint, but the potentially sizable feasible region will be left unexplored a considerable amount of time, causing overly biases to feasible points [14]. Therefore, the barrier approach explores near the edges of the feasible area inefficiently.

To overcome this, the authors in [18] introduced a different list for hyper-rectangles with infeasible centers. One such hyper-rectangle is chosen in each iteration, next to the traditional selection of potentially optimal hyper-rectangles (POH) [24]. However, the experimental investigation revealed that this modification does not give significant improvement.

2.2 **DIRECT** extension based on a neighborhood assignment strategy (NAS)

The second **DIRECT**-type extension is based on the neighborhood assignment strategy (NAS) and was also proposed in [18] (let us call this approach **DIRECT-NAS**). **DIRECT-NAS** assigns the value to an infeasible point based on the objective function values found in neighboring feasible points. This is achieved in the following way. A hyper-rectangle with an infeasible midpoint is expanded by doubling each side length while keeping the center point. Suppose the resulting hyper-rectangle contains at least one feasible midpoint. The surrogate function value $\hat{f}^{\min} + \epsilon |\hat{f}^{\min}|$ is assigned to an infeasible midpoint, where $\hat{f}^{\min}$ is the lowest objective function value and ϵ is the user's parameter. In [18] the proposed value to use is $\epsilon = 10^{-6}$. Suppose the enlarged hyper-rectangle does not contain any feasible points. In that case, **DIRECT-NAS** assigns $f^{\max} + \lambda$, where $f^{\max}$ is the largest objective function value found so far, and the proposed value for the parameter λ is 1. In such a way, a subdivision of hyper-rectangles with infeasible midpoints, close to the feasible region's edges, is based on the objective function values found in the neighborhood. **DIRECT-NAS** seems more promising compared to a barrier approach but adds extra calculations.

2.3 A modified **DIRECT** algorithm with a sub-dividing step

Another recent **DIRECT**-type proposal for the problem (1) is introduced in [32]. Here, the authors adopted two ideas from [18]. First, the authors incorporate the barrier strategy into the **DIRECT** framework by assigning a very large penalty value to hyper-rectangles with infeasible centers. Next, they also create a list of hyper-rectangles with infeasible midpoints, as proposed in [18]. After subdividing all traditional POHs, a new sub-dividing step is started, and all hyper-rectangles with infeasible midpoints are also subdivided. After this step, the minimum function value ($f^{\min}$) is updated. Let us call this approach **DIRECT-sub-div**. The authors showed that the sub-dividing step decomposes the boundaries of the hidden constraints and quite efficiently exposes the edges of the feasible region.

This approach's main concern is that the authors do not define when and how often this step should be activated and did not provide a sensitive analysis of the

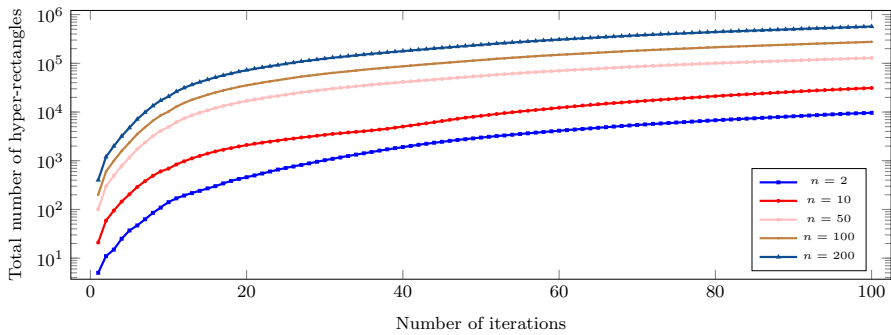


Fig. 1 The total number of partitions (hyper-rectangles) per iteration created with the DIRECT-GL algorithm on the test problem *TI* with varying dimensionality ($n = 2, 10, 50, 100$, and 200)

DIRECT-sub-div algorithm. Partitioning of large infeasible areas may create numerous infeasible hyper-rectangles. This situation is particularly relevant for higher dimensionality problems, as the total number of partitions (hyper-rectangles) per iteration increases sharply with the increase of dimension (see Fig. 1).

3 A new DIRECT-GLh algorithm for problems with hidden constraints

In this section, we present an extension of our recently introduced DIRECT-GL algorithm [57] for global optimization problems with hidden constraints. Like original DIRECT-GL, DIRECT-GLh starts by transforming a region D into the unit hypercube $\bar{D} = [0, 1]^n$. Let the partition of the unit hypercube $\bar{D}$ at iteration $k \geq 1$ be defined as

$$\mathcal{P}_k = \{\bar{D}_k^i : i \in \mathbb{I}_k\},$$

where $\bar{D}_k^i = [\mathbf{a}_k^i, \mathbf{b}_k^i] = \{\mathbf{x} \in \mathbb{R}^n : 0 \leq a_{k_j}^i \leq x_j \leq b_{k_j}^i \leq 1, j = 1, \dots, n, \forall i \in \mathbb{I}_k\}$, $\mathbb{I}_k$ is the index set identifying the current partition $\mathcal{P}_k$, and δ_k^i is a measure (distance, size) of the hyper-rectangle $\bar{D}_k^i$.

In the following two subsections, the main extensions to DIRECT-GL are described.

3.1 Feasibility detection

In [8,27,58] it was shown that finding at least one feasible point while solving general optimization problems often could be very costly. For problems with hidden constraints, finding a feasible point can be even more challenging, as analytic information about the constraints is unavailable. To deal with this, we employ the uniform partitioning of $\bar{D}$, as stated in Definition 1.

Definition 1 Consider $\bar{D}$, set $\mathbb{I}_k = \{1\}$, where $k = 1$, and perform the following steps:

- **Step 1** Find an index $j \in \mathbb{I}_k$ and a corresponding hyper-rectangle $\bar{D}_k^j$, such that

$$\bar{D}_k^j = \max_j \left\{ j = \arg \max_{i \in \mathbb{I}_k} \{ \delta_k^i \} \right\}. \quad (2)$$

- **Step 2** Subdivide (trisection) a hyper-rectangle $\bar{D}_k^j$ and check the feasibility of the problem (1) at midpoints of all new hyper-rectangles. Set $k = k + 1$, update $\mathbb{I}_k$ and δ_k^i for all new hyper-rectangles.
- **Step 3** If $D^{\text{feas}} = \emptyset$ **repeat** Steps 1-2; otherwise **terminate**.

At Step 1, a hyper-rectangle with the largest size is selected. If there are several such hyper-rectangles, the preference is given to a hyper-rectangle with the largest index value. At Step 2, the chosen hyper-rectangle is subdivided using a trisection strategy, and the feasibility of all new midpoints is checked. We continue this process until at least one feasible point is found, i.e., $D^{\text{feas}} \neq \emptyset$. After this, all effort is put into the improvement of the feasible solution.

3.2 Solution improvement

In the second extension, we first transform problem (1) to (3) as follows:

$$\begin{aligned} & \min_{\mathbf{x} \in D} \xi(\mathbf{x}, \mathbf{x}^{\min}, f^{\min}), \\ \xi(\mathbf{x}, \mathbf{x}^{\min}, f^{\min}) &= \begin{cases} f(\mathbf{x}), & \text{if } \mathbf{x} \in D^{\text{feas}} \\ f^{\min} + d(\mathbf{x}^{\min}, \mathbf{x}), & \text{otherwise,} \end{cases} \end{aligned} \quad (3)$$

where $f^{\min}$ is the best feasible objective function value found so far and $d(\mathbf{x}^{\min}, \mathbf{x})$ is Euclidean distance from the current best minimum point ($\mathbf{x}^{\min}$) to the point ($\mathbf{x}$):

$$d(\mathbf{x}^{\min}, \mathbf{x}) = \sqrt{\sum_{j=1}^n (x_j^{\min} - x_j)^2}. \quad (4)$$

When $\mathbf{x} \in D^{\text{feas}}$, the objective function is evaluated at $f(\mathbf{x})$, otherwise a value $f^{\min} + d(\mathbf{x}^{\min}, \mathbf{x})$ is assigned to an infeasible midpoint. This way, infeasible midpoints are assigned values based on the best currently found minimum value $f^{\min}$ and the distance from the current minimum point. Moreover, the convergence of DIRECT-GLh to a feasible solution point is assured, even when complicated shape constraints are faced. Note that this comes with a slight performance overhead. When a better $f^{\min}$ value is found, $f^{\min} + d(\mathbf{x}^{\min}, \mathbf{x})$ is updated for all hyper-rectangles with infeasible midpoints. A geometrical illustration of this procedure is shown in Fig. 2.

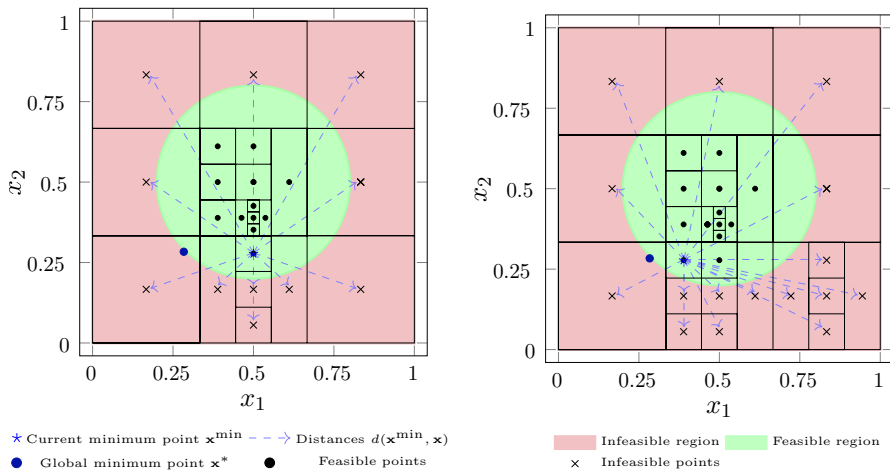


Fig. 2 Geometric illustration of DIRECT-GLh on the two-dimensional T1 test problem in the third (the left side), and in the fourth (the right side) iterations

The biggest issue with (3) is that values of the objective function and distances can differ significantly. To treat equally, we normalize them:

$$\begin{aligned}
 \tilde{f}(\mathbf{x}) &= \begin{cases} 1, & \text{if } f^{\max} == f^{\min} \\ \frac{f(\mathbf{x}) - f^{\min}}{f^{\max} - f^{\min}}, & \text{otherwise} \end{cases} \\
 \tilde{d}(\mathbf{x}^{\min}, \mathbf{x}) &= \frac{d(\mathbf{x}^{\min}, \mathbf{x}) - d^{\min}}{d^{\max} - d^{\min}}, \quad (5)
 \end{aligned}$$

where $d^{\max}$ is the length of the diagonal of the hypercube $\tilde{D}$ and $d^{\min} = 0$. Again, this adds some performance overhead, as the normalized objective function and distance values need to be updated each time the hyper-rectangle with the new largest/lowest objective value at the center point is found. By taking into account (5) and (3), we obtain:

$$\begin{aligned}
 &\min_{\mathbf{x} \in D} \tilde{\xi}(\mathbf{x}, \mathbf{x}^{\min}, \tilde{f}^{\min}), \\
 \tilde{\xi}(\mathbf{x}, \mathbf{x}^{\min}, \tilde{f}^{\min}) &= \begin{cases} \tilde{f}(\mathbf{x}), & \text{if } \mathbf{x} \in D^{\text{feas}}, \\ \tilde{f}^{\min} + \tilde{d}(\mathbf{x}^{\min}, \mathbf{x}), & \text{otherwise.} \end{cases} \quad (6)
 \end{aligned}$$

Note that $\tilde{f}^{\min} = 0$, therefore could be omitted from (6).

3.3 Algorithmic steps

The detailed description of the DIRECT-GLh algorithm is given in Algorithm 1. Inputs are the problem (P) and stopping conditions: tolerance (ε_{pe}), the maximal number

Algorithm 1: Pseudo code of the DIRECT-GLh algorithm

```

input :  $P, \varepsilon_{pe}, M^{\max}, K^{\max}$ ;
output:  $f^{\min}, \mathbf{x}^{\min}, pe, k, m$ ;

1 Initialize  $k = 1, m = 1, \mathbb{I}_k = \{1\}$ ;
2 if  $\mathbf{x}^1 \notin D^{\text{feas}}$  then
3   Find a hyper-rectangle with a feasible center point; // using Definition 1
4   Update  $f^{\min}, \mathbf{x}^{\min}, pe, m, \mathbb{I}_k$ ;
5 else
6   Set  $f^{\min} = f(\mathbf{x}^1), \mathbf{x}^{\min} = \mathbf{x}^1$  and update  $pe$ ;
7 end
8 Calculate  $d(\mathbf{x}^{\min}, \mathbf{x}^i), \xi(\mathbf{x}, \mathbf{x}^{\min}, f^{\min})$  and  $\tilde{\xi}(\mathbf{x}, \mathbf{x}^{\min}, \tilde{f}^{\min}), \mathbf{x}^i \in \bar{D}_k^i, \forall i \in \mathbb{I}_k$ ; //  $d$  (4),  $\xi$  (3),  $\tilde{\xi}$  (6)
9 while  $pe > \varepsilon_{pe}$  and  $m < M^{\max}$  and  $k < K^{\max}$  do //  $pe$  defined in (8)
10  Identify the “global” (index) set  $\mathbb{G}_k \subseteq \mathbb{I}_k$  of POHs on  $\tilde{\xi}(\mathbf{x}, \mathbf{x}^{\min}, \tilde{f}^{\min})$ ; // using Definition 2
    from [57]
11  Identify “local” (index) set  $\mathbb{L}_k \subseteq \mathbb{I}_k$  of POHs on  $d(\mathbf{x}^{\min}, \mathbf{x}^i)$ ; // using Definition 3 from [57]
12  Find the unique union of these two sets  $\mathbb{S}_k = \mathbb{G}_k \cup \mathbb{L}_k$ ;
13  foreach  $i \in \mathbb{S}_k$  do
14    Subdivide (trisect) hyper-rectangle  $\bar{D}_k^i$ , update  $\mathbb{I}_k$ ;
15    Calculate  $\xi(\mathbf{x}, \mathbf{x}^{\min}, f^{\min})$  at new center points; //  $\xi$  defined in (3)
16    Update  $f^{\min}, \mathbf{x}^{\min}, pe$ , and  $m$ ;
17  end
18  if  $\|\mathbf{x}_k^{\min} - \mathbf{x}_{k-1}^{\min}\| \geq 10^{-6}$  then
19    Calculate  $d(\mathbf{x}^{\min}, \mathbf{x}^i), \xi(\mathbf{x}, \mathbf{x}^{\min}, f^{\min})$  and  $\tilde{\xi}(\mathbf{x}, \mathbf{x}^{\min}, \tilde{f}^{\min}), \mathbf{x}^i \in \bar{D}_k^i, \forall i \in \mathbb{I}_k$ ;
20  else
21    Calculate  $\tilde{\xi}(\mathbf{x}, \mathbf{x}^{\min}, \tilde{f}^{\min}), \mathbf{x}^i \in \bar{D}_k^i$  for all  $i = m^{\text{old}}, \dots, m^{\text{new}}$ ;
22  end
23  Increase  $k = k + 1$ ;
24 end
25 return  $f^{\min}, \mathbf{x}^{\min}, pe, k, m$ ;

```

of function evaluations ($M^{\max}$), and the maximal number of iterations ($K^{\max}$). After termination, DIRECT-GLh returns the best objective function value $f^{\min}$, solution point $\mathbf{x}^{\min}$, and performance measures: percent error (pe), the total number of function evaluations (m), and the total number of iterations (k). The algorithm starts from the single center point. If it is infeasible, the algorithm performs uniform sampling until a feasible point is found (see Lines 2–7 in Algorithm 1). DIRECT-GLh uses the two-step based strategy for selecting POHs (sets $\mathbb{G}_k$ and $\mathbb{L}_k$), as was introduced in [57]. However, the original DIRECT-GL algorithm separately handles $\mathbb{G}_k$ and $\mathbb{L}_k$ sets. In DIRECT-GLh, $\mathbb{G}_k$ and $\mathbb{L}_k$ are combined into a unique union set $\mathbb{S}_k$ (see Lines 10–12 in Algorithm 1). Then, DIRECT-GLh subdivides all hyper-rectangles from the set $\mathbb{S}_k$, calculates $\xi(\mathbf{x}, \mathbf{x}^{\min}, f^{\min})$, and updates $f^{\min}$ and $\mathbf{x}^{\min}$ if needed (see Lines 13–17). If the new best minimum point is found, the algorithm performs 19, otherwise 21. DIRECT-GLh repeats these steps until the stopping condition is not satisfied.

3.4 Convergence analysis

The convergence properties of DIRECT-type algorithms are broadly investigated (see, e.g., [15,20,24,33,34,47]). The developed DIRECT-GLh algorithm is an extension of DIRECT-GL, based on the “Divide the best” strategy [45] and has the everywhere-dense type of convergence. In general, DIRECT-GL and other DIRECT-

type algorithms for box-constrained problems cannot guarantee convergence for problems with hidden constraints. If hidden constraints are handled by returning NaN (when f is called at $\mathbf{x} \notin D^{\text{feas}}$), this situation will cause an undefined behavior for all box-constrained DIRECT-type algorithms. Moreover, suppose the initial midpoint is infeasible, i.e., $\mathbf{x}^1 \notin D^{\text{feas}}$. These algorithms will then be stuck at this point. The selection of potentially optimal hyper-rectangles will not select this hyper-rectangle for further subdivision and cause an undefined state of the algorithm. To avoid this, algorithmic modifications and extensions are needed, as discussed in Sect. 2.

The DIRECT-GLh algorithm guarantees converge to the global minimum $\mathbf{x}^*$ if $D^{\text{feas}} \neq \emptyset$ and the objective function is at least continuous in the neighborhood of $\mathbf{x}^*$. First, DIRECT-GLh employs the feasibility detection step, which is performed until at least one feasible point is detected. During this step, DIRECT-GLh does not have any problems dealing with NaN values. Moreover, as we do not have any information about the hidden constraints, the search space's uniform partitioning is a logical sampling strategy. Finally, independently of how small is the feasible region, we can show that if $D^{\text{feas}} \neq \emptyset$, this procedure is finite, and the feasible solution will be found when the maximal allowed number of generated trial points $M^{\text{max}} \rightarrow \infty$.

Proposition 1 *For any feasible point $\mathbf{x} \in D^{\text{feas}}$ and any $\epsilon > 0$ there exist an iteration number $k(\epsilon) \geq 1$ and a point $\mathbf{x}^i \in \bar{D}_k^i \subset \bar{D}$, such that $\|\mathbf{x}^i - \mathbf{x}\| < \epsilon$.*

Proof During the feasibility detection step DIRECT-GLh performs the uniform partitioning of $\bar{D}$ by selecting and trisecting the hyper-rectangle from a group having the largest diameter δ_k^{max} (see Definition 1). In such a way, the volume of the hyper-rectangle is reduced. Since the number of hyper-rectangles with the largest diameter is finite, after a sufficiently large number of iterations, $k > k(\epsilon)$, all the hyper-rectangles with the current maximal diameter δ_k^{max} will be partitioned. Suppose $\mathbf{x} \in D^{\text{feas}}$ was not found yet. In that case, such a procedure will be repeated with a new group of the largest hyper-rectangles until the largest hyper-rectangle of the current partition will have a diameter smaller than ϵ . $\square$

Note that here everywhere dense points are uniformly sampled in the search space $\bar{D}$ as we do not have any analytical information about the hidden constraints. In the solution improvement step (see Sect. 3.2), the main extension to ensure the convergence is an assignment of the value $f^{\min} + d(\mathbf{x}^{\min}, \mathbf{x})$ to an infeasible midpoint $\mathbf{x} \in D^{\text{hidden}}$. Other modifications are needed only to speed up the convergence. Therefore, the feasibility detection and defined behavior of infeasible points assure everywhere dense type of convergence of DIRECT-GLh, which follows from the convergence of DIRECT-GL. Note that at this phase, the everywhere dense nature of the sampled points does not mean that they are uniformly located on the search space. This can be seen in Fig. 2, where the highest concentration of sampled points is observed in the vicinity of the global minimum $\mathbf{x}^*$.

4 Results and discussion

4.1 Experimental investigation on the DIRECTLib test set

In this section, we investigate the efficiency of the developed DIRECT-GLh algorithm and compare it with three known DIRECT-type extensions for problem (1). Test problems from the most recent version of DIRECTLib [59] (67 in total) are used to evaluate selected algorithms' performance. We assume that any information about the constraint functions is unavailable. In the experimental setup, the hidden search area (D^{hidden}) is defined as

$$D^{\text{hidden}} = \{\mathbf{x} \in D : \mathbf{g}(\mathbf{x}) \leq 0, \mathbf{h}(\mathbf{x}) = 0\}. \quad (7)$$

Still, this information is unavailable for algorithms and used only to determine whether a certain point is feasible or not. Since the global minima f^* are known, tested algorithms were stopped either when a point $\mathbf{x}$ was generated such that the percent error (pe)

$$pe = 100 \times \begin{cases} \frac{f(\mathbf{x}) - f^*}{|f^*|}, & f^* \neq 0, \\ f(\mathbf{x}), & f^* = 0, \end{cases} \quad (8)$$

is smaller than the tolerance value (ε_{pe}), i.e., $pe \leq \varepsilon_{pe}$. In our investigation, $\varepsilon_{pe} = 10^{-2}$ was used. Additionally, we set the maximal number of function evaluations to 10^6 and the maximal execution time to six hours. When the algorithm exceeded any of these conditions, the final result was set to 10^6 . All computations were carried out on a 6-core computer with 8th Generation Intel R Core™ i7-8750H @ 2.20GHz Processor, 16 GB of RAM, and MATLAB R2020a software environment.

The authors of DIRECT-sub-div did not provide public access to their implementation of the algorithm. Moreover, the detailed information when an extra subdividing step should be performed is not specified. In our implementation of DIRECT-sub-div, the subdividing step is controlled by two parameters λ and N , and performed on iterations $k = \lambda^N$, $N = 1, 2, \dots$. We tested the algorithm performance when λ is equal to 2 and 5. The obtained experimental results with all tested algorithms are summarized in Tables 1, 2. Here, the best results are given in bold. In both tables, in the first column (Label), the problem's name is listed, while in the second – dimensionality of the problem (n). In the third column (Constr. type), we specify the type of hidden constraints: linear (L) or nonlinear (NL). Next, in consecutive columns, the total number of function evaluations (f_{eval}) and the total execution time (in seconds) in Table 2 are given.

First, we consider the situation when finding at least one feasible point is costly (see Sect. 3.1 for more information on this). For this, from DIRECTLib [59], we have selected a subset of test problems having a tiny feasible region (D^{feas}). DIRECT-GLh is the fastest to locate the first feasible point for six out of the nine test problems (see Table 1). Moreover, the average number of function evaluations is more than two times smaller than the second-best algorithm DIRECT-NAS.

Next, solving all 67 test instances (see Table 2) from DIRECTLib, DIRECT-GLh fails to solve (5/67) problems, while the second-best DIRECT-NAS fails to solve

Table 1 The total number of function evaluations needed by algorithms to find the first feasible point ($\mathbf{x} \in D^{\text{feas}}$) on a selected subset of test problems from DIRECTLib

Label	n	Constr. type	DIRECT-GLH	DIRECT-NAS	DIRECT-Barrier	DIRECT-sub-div ($\lambda = 2$)	DIRECT-sub-div ($\lambda = 5$)
Bunnag 6	10	L	136,549	363,929	1,240,029	5,614,449	1,951,629
Genocop 9	4	L	8939	7,543	10,935	42,261	53,695
Genocop 11	4	L	2,889	7917	9477	73,881	36,549
G06	2	NL	33,211	39,063	59,049	202,589	136,125
G16	5	NL	803	2513	2673	39,253	25,203
hs021mod	7	L	33,051	33,309	85,293	127,569	54,435
P2d	5	NL	139	137	243	81	231
P9	3	L	81	51	189	225	99
s232	2	L	817	2817	3645	10,449	3681
Average (overall)			24,053	50,809	156,837	678,973	251,294

(15/67). Naturally, the low-dimensional test problems are the simplest of all tested approaches, but still DIRECT-GLh requires approximately 1.3 times fewer function evaluations compared to DIRECT-NAS. However, the efficiency of DIRECT-NAS decreases significantly solving more complicated problems, i.e., larger dimensionality, and when D^{hidden} is nonlinear. For such problems, DIRECT-GLh requires approximately 2.4 times fewer function evaluations compared to the second best DIRECT-NAS. Experimental results did not reveal a significant advantage of DIRECT-sub-div over DIRECT-Barrier. According to authors [32], the subdividing step should help decompose the edge of hidden constraints more efficiently. Still, solving higher-dimension problems, the subdividing step's execution often takes a considerable amount of time and uses a substantial budget of function evaluations. Therefore, while the average (and the median) number of function evaluations is smaller, the total number of failed to solve problems is smaller using DIRECT-Barrier.

4.2 Experimental investigation on Emmental-type GKLS test functions

Additionally, we compare the performance of the selected algorithms on a different set of test problems. Here, we consider the Emmental-type GKLS test function generator [51], which can generate multi-dimensional constrained test problems. The primary purpose of this generator is to benchmark continuous, constrained global optimization methods. However, by employing the same testing strategy as in Sect. 4.1, we can use it to test algorithms for problems with hidden constraints.

We consider ten classes of Emmental-type GKLS test problems (see Table 3). Classes of growing dimensionality ($n = 2, \dots, 6$) and different complexity – “simple” and “hard” have been considered. Ten test functions for each class have been generated. The global minimizer for these problems is always located on the boundary of the feasible region. The number of local minima is set to 30, and the global minimum value $f^* = -1.0$ was used for all classes. These are the default settings for the Emmental-type GKLS generator [51]. The number of constraints is 32 (20, 2, and 10 for each of three possible types considered by Emmental-type GKLS generator), with 5 active constraints at the global minimum [3,51].

In these experiments, $\varepsilon_{\text{pe}} = 1$ was used, while the other stopping conditions were the same as in the previous experiments. Algorithms were compared using a number of function evaluation criteria. The average, median, and maximal number of function evaluations for each class are reported in Table 4. The notation “> 1, 000, 000 (j)” means that the method under consideration was not able to solve j problems using a budget of 1, 000, 000 function evaluations.

First, it can be seen that DIRECT-GLh and DIRECT-NAS show the most promising results. For low dimensional ($n \leq 3$) test problems DIRECT-NAS has the best (lowest) average, median, and maximal function evaluations. The selection procedure used in the DIRECT-GLh algorithm selects a more extensive set of potential optimal hyper-rectangles per iteration [57,58,60]. Therefore, a more significant number of function evaluations is needed for cheaper problems. For more challenging problems ($n \geq 4$), the developed DIRECT-GLh almost always (except problems

Table 3 Description of Emmental-type GKLS test classes used in numerical experiments

Class	Difficulty	n	f^*	d	r	Number of Local minima	1-type $g(\mathbf{x})$	2-type $g(\mathbf{x})$	3-type $g(\mathbf{x})$	Active $g(\mathbf{x})$
1	Simple	2	-1	0.90	0.25	30	20	2	10	5
2	Hard	2	-1	0.90	0.15	30	20	2	10	5
3	Simple	3	-1	0.66	0.25	30	20	2	10	5
4	Hard	3	-1	0.90	0.25	30	20	2	10	5
5	Simple	4	-1	0.66	0.25	30	20	2	10	5
6	Hard	4	-1	0.90	0.25	30	20	2	10	5
7	Simple	5	-1	0.66	0.30	30	20	2	10	5
8	Hard	5	-1	0.66	0.25	30	20	2	10	5
9	Simple	6	-1	0.66	0.30	30	20	2	10	5
10	Hard	6	-1	0.60	0.25	30	20	2	10	5

Table 4 Numerical results on 10 Enmental-type GKLS test classes

Class	DIRECT-GLh	DIRECT-NAS	DIRECT-Barrier	DIRECT-sub-div($\lambda = 2$)	DIRECT-sub-div($\lambda = 5$)
Average number of function evaluations					
1	1,003	665	75,114	> 385,470	52,788
2	4,169	1,291	16,695	> 224,155	6,855
3	3,836	1,768	> 312,456	> 323,883	> 115,646
4	15,354	2,945	> 318,572	> 437,133	> 205,326
5	12,250	14,160	> 408,927	> 732,058	> 310,678
6	159,246	25,767	> 547,700	> 878,711	> 457,701
7	11,682	> 114,905	> 576,382	> 738,505	61,580
8	107,607	> 135,361	> 453,394	> 915,213	> 196,445
9	29,727	> 245,537	> 444,755	> 850,054	> 392,514
10	93,893	> 220,680	> 543,932	> 905,765	> 556,903
Median number of function evaluations					
1	882	628	2,842	18,963	941
2	2,489	833	11,110	4,788	2,681
3	2,204	1,185	33,957	30,930	7,570
4	3,499	2,331	172,798	96,607	8,618
5	1,375	8,984	25,353	965,845	14,706
6	16,075	16,190	584,914	1,000,000	353,895
7	8,075	13,984	775,569	903,643	28,855
8	10,649	26,917	160,699	1,000,000	145,176
9	12,077	77,460	134,525	1,000,000	219,296
10	15,513	31,600	596,751	1,000,000	651,424

Table 4 continued

Class	DIRECT-GLh	DIRECT-NAS	DIRECT-Barrier	DIRECT-sub-div($\lambda = 2$)	DIRECT-sub-div($\lambda = 5$)
Maximal number of function evaluations					
1	1,925	1,291	320,527	> 1,000,000 (3)	399,477
2	13,021	3,153	44,257	> 1,000,000 (2)	30,879
3	12,605	6,085	> 1,000,000 (2)	> 1,000,000 (3)	> 1,000,000 (1)
4	109,183	6,655	> 1,000,000 (2)	> 1,000,000 (4)	> 1,000,000 (2)
5	94,889	58,375	> 1,000,000 (4)	> 1,000,000 (5)	> 1,000,000 (3)
6	885,495	104,755	> 1,000,000 (5)	> 1,000,000 (8)	> 1,000,000 (3)
7	33,339	> 1,000,000 (1)	> 1,000,000 (5)	> 1,000,000 (5)	178,715
8	593,729	> 1,000,000 (1)	> 1,000,000 (4)	> 1,000,000 (9)	> 1,000,000 (1)
9	165,525	> 1,000,000 (2)	> 1,000,000 (4)	> 1,000,000 (8)	> 1,000,000 (3)
10	552,931	> 1,000,000 (2)	> 1,000,000 (4)	> 1,000,000 (9)	> 1,000,000 (5)

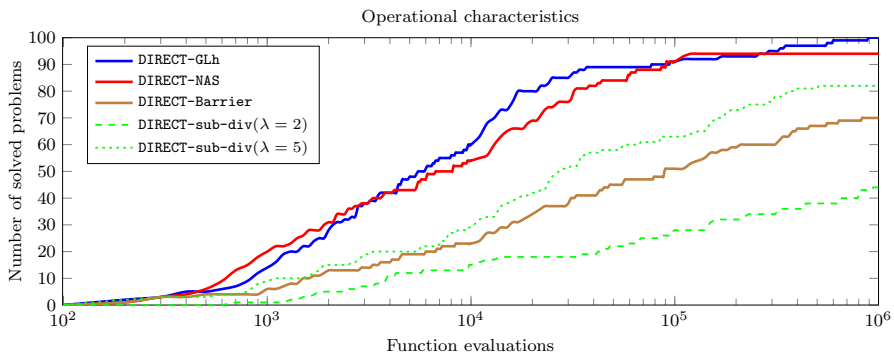


Fig. 3 Operational characteristics for the algorithms on Emmental-type GKLS test problems

Additionally, the operational characteristics [21,61] showing the behavior of the five tested algorithms on the whole set of 100 test problems are reported in Fig. 3. Operational characteristics provide the number of problems that can be solved with a given budget of function evaluations. The graphs reveal that DIRECT-GLh and DIRECT-NAS are two of the best performing algorithms. DIRECT-NAS is the best performing algorithm when a given budget of function evaluations is low. By increasing the budget of function evaluations, which is needed for larger and more challenging problems, DIRECT-GLh becomes a leader.

5 Conclusions

This paper proposes a new DIRECT-type algorithm (DIRECT-GLh) for global optimization problems with hidden constraints has been proposed. Two new techniques for faster detection of feasible points and more efficient convergence, especially for problems having solutions on the border of the unknown (hidden) feasible region, were implemented in DIRECT-GLh. Extensive numerical experiments executed on 67 problems from the current release of DIRECTLib and 100 Emmental-type GKLS test problems have revealed the promising performance of DIRECT-GLh with respect to other DIRECT-type extensions for optimization problems with hidden constraints.

Declarations

Data statement Data underlying this article can be accessed on Zenodo at <https://dx.doi.org/10.5281/zenodo.1218980>, and used under the Creative Commons Attribution license.

References

1. Bachoc, F., Helbert, C., Picheny, V.: Gaussian process optimization with failures: classification and convergence proof. *J. Glob. Opt.* **78**(3), 483–506 (2020). <https://doi.org/10.1007/s10898-020-00920-0>

2. Bartholomew-Biggs, M.C., Parkhurst, S.C., Wilson, S.P.: Using DIRECT to solve an aircraft routing problem. *Comput. Opt. Appl.* **21**(3), 311–323 (2002). <https://doi.org/10.1023/A:1013729320435>
3. Candelieri, A.: Sequential model based optimization of partially defined functions under unknown constraints. *J. Glob. Opt.* (2019). <https://doi.org/10.1007/s10898-019-00860-4>
4. Carter, R.G., Gablonsky, J.M., Patrick, A., Kelley, C.T., Eslinger, O.J.: Algorithms for noisy problems in gas transmission pipeline optimization. *Opt. Eng.* **2**(2), 139–157 (2001). <https://doi.org/10.1023/A:1013123110266>
5. Characklis, G.W., Kirsch, B.R., Ramsey, J., Dillard, K.E., Kelley, C.T.: Developing portfolios of water supply transfers. *Water Resour. Res.* (2006). <https://doi.org/10.1029/2005WR004424>
6. Chen, X., Kelley, C.T.: Optimization with hidden constraints and embedded Monte Carlo computations. *Opt. Eng.* **17**(1), 157–175 (2016). <https://doi.org/10.1007/s11081-015-9302-1>
7. Choi, T.D., Eslinger, O.J., Kelley, C.T., David, J.W., Etheridge, M.: Optimization of Automotive Valve Train Components with Implicit Filtering. *Opt. Eng.* **1**(1), 9–27 (2000). <https://doi.org/10.1023/A:1010071821464>
8. Costa, M.F.P., Rocha, A.M.A.C., Fernandes, E.M.G.P.: Filter-based direct method for constrained global optimization. *J. Glob. Opt.* **71**(3), 517–536 (2018). <https://doi.org/10.1007/s10898-017-0596-8>
9. David, J.W., Kelley, C.T., Cheng, C.Y.: Use of an implicit filtering algorithm for mechanical system parameter identification. *SAE Technical Papers* (1996). <https://doi.org/10.4271/960358>
10. Di Pillo, G., Liuzzi, G., Lucidi, S., Piccialli, V., Rinaldi, F.: A DIRECT-type approach for derivative-free constrained global optimization. *Comput. Opt. Appl.* **65**(2), 361–397 (2016). <https://doi.org/10.1007/s10589-016-9876-3>
11. Di Pillo, G., Lucidi, S., Rinaldi, F.: An approach to constrained global optimization based on exact penalty functions. *J. Opt. Theory Appl.* **54**(2), 251–260 (2010). <https://doi.org/10.1007/s10898-010-9582-0>
12. Di Serafino, D., Liuzzi, G., Piccialli, V., Riccio, F., Toraldo, G.: A modified Dividing RECTangles algorithm for a problem in astrophysics. *J. Opt. Theory Appl.* **151**(1), 175–190 (2011). <https://doi.org/10.1007/s10957-011-9856-9>
13. Donskoi, V.I.: Partially defined optimization problems: An approach to a solution that is based on pattern recognition theory. *J. Soviet Mat.* (1993). <https://doi.org/10.1007/BF01097516>
14. Finkel, D.E.: Global optimization with the DIRECT algorithm. Ph.D. thesis, North Carolina State University (2005)
15. Finkel, D.E., Kelley, C.T.: Additive scaling and the DIRECT algorithm. *J. Glob. Opt.* **36**(4), 597–608 (2006). <https://doi.org/10.1007/s10898-006-9029-9>
16. Fletcher, R.: *Practical Methods of Optimization*, 2nd edn. John and Sons Chichester, United Kingdom (1987)
17. Forrester, A.I.J., Keane, A.J.: Recent advances in surrogate-based optimization. *Prog. Aerospace Sci.* **45**(1), 50–79 (2009). <https://doi.org/10.1016/j.paerosci.2008.11.001>
18. Gablonsky, J.M.: Modifications of the DIRECT algorithm. Ph.D. thesis, North Carolina State University (2001)
19. Gablonsky, J.M., Kelley, C.T.: A locally-biased form of the DIRECT algorithm. *J. Glob. Opt.* **21**(1), 27–37 (2001). <https://doi.org/10.1023/A:1017930332101>
20. Gorodetsky, S.: Diagonal Generalization of the DIRECT Method for Problems with Constraints. *Autom. Remote Contr.* **81**(8), 1431–1449 (2020). <https://doi.org/10.1134/S0005117920080068>
21. Grishagin, V.A.: Operating characteristics of some global search algorithms. In: *Problems of Stochastic Search*, vol. 7, pp. 198–206. Zinatne, Riga (1978). In Russian
22. Jones, D.R.: The Direct global optimization algorithm. In: Floudas, C.A., Pardalos, P.M. (eds.) *The Encyclopedia of Optimization*, pp. 431–440. Kluwer Academic Publishers, Dordrecht (2001)
23. Jones, D.R., Martins, J.R.R.A.: The DIRECT algorithm: 25 years later. *J. Glob. Opt.* (2020). <https://doi.org/10.1007/s10898-020-00952-6>
24. Jones, D.R., Perttunen, C.D., Stuckman, B.E.: Lipschitzian optimization without the Lipschitz constant. *J. Opt. Theory Appl.* **79**(1), 157–181 (1993). <https://doi.org/10.1007/BF00941892>
25. Kirsch, B.R., Characklis, G.W., Dillard, K.E., Kelley, C.T.: More efficient optimization of long-term water supply portfolios. *Water Resources Research* (2009). <https://doi.org/10.1029/2008WR007018>
26. Le Digabel, S., Wild, S.M.: A Taxonomy of Constraints in Simulation-Based Optimization. *arXiv e-prints* [arXiv:1505.07881](https://arxiv.org/abs/1505.07881) (2015)

27. Liu, H., Xu, S., Chen, X., Wang, X., Ma, Q.: Constrained global optimization via a direct-type constraint-handling technique and an adaptive metamodeling strategy. *Struct. Multidisc. Opt.* **55**(1), 155–177 (2017). <https://doi.org/10.1007/s00158-016-1482-6>
28. Liu, Q., Zeng, J., Yang, G.: MrDIRECT: a multilevel robust DIRECT algorithm for global optimization problems. *J. Glob. Opt.* **62**(2), 205–227 (2015). <https://doi.org/10.1007/s10898-014-0241-8>
29. Liuzzi, G., Lucidi, S., Piccialli, V.: A DIRECT-based approach exploiting local minimizations for the solution of large-scale global optimization problems. *Comput. Opt. Appl.* **45**, 353–375 (2010). <https://doi.org/10.1007/s10589-008-9217-2>
30. Liuzzi, G., Lucidi, S., Piccialli, V.: A partition-based global optimization algorithm. *J. Glob. Opt.* **48**(1), 113–128 (2010). <https://doi.org/10.1007/s10898-009-9515-y>
31. Mockus, J., Paulavičius, R., Rusakevičius, D., Šešok, D., Žilinskas, J.: Application of Reduced-set Pareto-Lipschitzian Optimization to truss optimization. *J. Glob. Opt.* **67**(1–2), 425–450 (2017). <https://doi.org/10.1007/s10898-015-0364-6>
32. Na, J., Lim, Y., Han, C.: A modified DIRECT algorithm for hidden constraints in an LNG process optimization. *Energy* p. 488–500 (2017). <https://doi.org/10.1016/j.energy.2017.03.047>
33. Paulavičius, R., Chiter, L., Žilinskas, J.: Global optimization based on bisection of rectangles, function values at diagonals, and a set of Lipschitz constants. *J. Glob. Opt.* **71**(1), 5–20 (2018). <https://doi.org/10.1007/s10898-016-0485-6>
34. Paulavičius, R., Sergeyev, Y.D., Kvasov, D.E., Žilinskas, J.: Globally-biased DISIMPL algorithm for expensive global optimization. *J. Glob. Opt.* **59**(2–3), 545–567 (2014). <https://doi.org/10.1007/s10898-014-0180-4>
35. Paulavičius, R., Sergeyev, Y.D., Kvasov, D.E., Žilinskas, J.: Globally-biased BIRECT algorithm with local accelerators for expensive global optimization. *Expert Syst. Appl.* **144**, 113052 (2020). <https://doi.org/10.1016/j.eswa.2019.113052>
36. Paulavičius, R., Žilinskas, J.: Analysis of different norms and corresponding Lipschitz constants for global optimization. *Technol. Econ. Develop. Econ.* **36**(4), 383–387 (2006). <https://doi.org/10.1080/13928619.2006.9637758>
37. Paulavičius, R., Žilinskas, J.: Analysis of different norms and corresponding Lipschitz constants for global optimization in multidimensional case. *Inf. Technol. Cont.* **36**(4), 383–387 (2007)
38. Paulavičius, R., Žilinskas, J.: Simplicial Lipschitz optimization without the Lipschitz constant. *J. Glob. Opt.* **59**(1), 23–40 (2013). <https://doi.org/10.1007/s10898-013-0089-3>
39. Paulavičius, R., Žilinskas, J.: *Simplicial Global Optimization*. SpringerBriefs in Optimization. Springer berlin (2014). <https://doi.org/10.1007/978-1-4614-9093-7>
40. Paulavičius, R., Žilinskas, J.: Advantages of simplicial partitioning for Lipschitz optimization problems with linear constraints. *Opt. Let.* **10**(2), 237–246 (2016). <https://doi.org/10.1007/s11590-014-0772-4>
41. Pintér, J.D.: *Global optimization in action: continuous and Lipschitz optimization: algorithms, implementations and applications*, Nonconvex Optimization and Its Applications, vol. 6. Springer US, Berlin (1996). <https://doi.org/10.1007/978-1-4757-2502-5>
42. Piyavskii, S.A.: An algorithm for finding the absolute minimum of a function. *Theory Opt. Solut.* **2**, 13–24 (1967). [https://doi.org/10.1016/0041-5553\(72\)90115-2](https://doi.org/10.1016/0041-5553(72)90115-2). (In Russian)
43. Rios, L.M., Sahinidis, N.V.: Derivative-free optimization: a review of algorithms and comparison of software implementations. *J. Glob. Opt.* **56**(3), 1247–1293 (2007). <https://doi.org/10.1007/s10898-012-9951-y>
44. Rudenko, L.I.: Objective functional approximation in a partially defined optimization problem. *J. Mat. Sci.* **72**(5), 3359–3363 (1994). <https://doi.org/10.1007/BF01261697>
45. Sergeyev, Y.D.: On convergence of divide the best global optimization algorithms. *Optimization* **44**(3), 303–325 (1998)
46. Sergeyev, Y.D., Candelieri, A., Kvasov, D.E., Perego, R.: Safe global optimization of expensive noisy black-box functions in the δ -Lipschitz framework. *Soft Computing* **24**(23), 17715–17735 (2020). <https://doi.org/10.1007/s00500-020-05030-3>
47. Sergeyev, Y.D., Kvasov, D.E.: Global search based on diagonal partitions and a set of Lipschitz constants. *SIAM J. Opt.* **16**(3), 910–937 (2006). <https://doi.org/10.1137/040621132>
48. Sergeyev, Y.D., Kvasov, D.E.: Lipschitz global optimization. In: Cochran, J.J., Cox, L.A., Keskinocak, P., Kharoufeh, J.P., Smith, J.C. (eds.) *Wiley Encyclopedia of Operations Research and Management Science* (in 8 volumes), vol. 4, pp. 2812–2828. Wiley, New York (2011)

49. Sergeyev, Y.D., Kvasov, D.E.: *Deterministic Global Optimization: An Introduction to the Diagonal Approach*. SpringerBriefs in Optimization. Springer (2017). <https://doi.org/10.1007/978-1-4939-7199-2>
50. Sergeyev, Y.D., Kvasov, D.E., Khalaf, F.M.H.: A one-dimensional local tuning algorithm for solving GO problems with partially defined constraints. *Opt. Let.* **1**(1), 85–99 (2007). <https://doi.org/10.1007/s11590-006-0015-4>
51. Sergeyev, Y.D., Kvasov, D.E., Mukhametzhanov, M.S.: Emmental-Type GKLS-Based Multiextremal Smooth Test Problems with Non-linear Constraints. In: Battiti, R., Kvasov, D.E., Sergeyev, Y.D. (eds.) *Learn. Intel. Opt.*, pp. 383–388. Springer International Publishing, Cham (2017)
52. Sergeyev, Y.D., Pugliese, P., Famularo, D.: Index information algorithm with local tuning for solving multidimensional global optimization problems with multiextremal constraints. *Mat. Prog.* **96**(3), 489–512 (2003). <https://doi.org/10.1007/s10107-003-0372-z>
53. Shan, S., Wang, G.G.: Metamodeling for high dimensional simulation-based design problems. *J. Mech. Des.* **132**(5), 051009 (2010). <https://doi.org/10.1115/1.4001597>
54. Shan, S., Wang, G.G.: Survey of modeling and optimization strategies to solve high-dimensional design problems with computationally-expensive black-box functions. *Struct. Multidis. Opt.* **41**(2), 219–241 (2010). <https://doi.org/10.1007/s00158-009-0420-2>
55. Shubert, B.O.: A sequential method seeking the global maximum of a function. *SIAM J. Num. Anal.* **9**, 379–388 (1972). <https://doi.org/10.1137/0709036>
56. Stoneking, D.E., Bilbro, G.L., Gilmore, P.A., Trew, R.J., Kelley, C.T.: Yield optimization using a gaas process simulator coupled to a physical device model. *IEEE Trans. Microwave Theory Techniq.* **40**(7), 1353–1363 (1992). <https://doi.org/10.1109/22.146318>
57. Stripinis, L., Paulavičius, R., Žilinskas, J.: Improved scheme for selection of potentially optimal hyperrectangles in DIRECT. *Opt. Let.* **12**(7), 1699–1712 (2018). <https://doi.org/10.1007/s11590-017-1228-4>
58. Stripinis, L., Paulavičius, R., Žilinskas, J.: Penalty functions and two-step selection procedure based DIRECT-type algorithm for constrained global optimization. *Struct. Multidis. Opt.* **59**(6), 2155–2175 (2019). <https://doi.org/10.1007/s00158-018-2181-2>
59. Stripinis, L., Paulavičius, R.: DIRECTLib – a library of global optimization problems for DIRECT-type methods, v1.2 (2020). <https://doi.org/10.5281/zenodo.3948890>
60. Stripinis, L., Žilinskas, J., Casado, L.G., Paulavičius, R.: On MATLAB experience in accelerating DIRECT-GLce algorithm for constrained global optimization through dynamic data structures and parallelization. *Appl. Mat. Comput.* **390**, 125596 (2021). <https://doi.org/10.1016/j.amc.2020.125596>
61. Strongin, R.G., Sergeyev, Y.D.: *Global Optimization with Non-Convex Constraints: Sequential and Parallel Algorithms*. Kluwer Academic Publishers, Dordrecht (2000)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.